

Universal Scaling of Identified Particle Spectra with ALICE Data

Thiago Siqueira Domingues

May 13, 2026

Abstract

This document explains the analysis workflow implemented in a Jupyter notebook that reads publicly available ALICE HEPData tables for Pb–Pb collisions at $\sqrt{s_{NN}} = 2.76$ TeV and Xe–Xe collisions at $\sqrt{s_{NN}} = 5.44$ TeV. The goal is to construct the universal scaling function $U(x_T)$ for pions, kaons, and protons, defined as

$$x_T = \frac{p_T}{\langle p_T \rangle}, \quad U(x_T) = \frac{\langle p_T \rangle}{N} \frac{dN}{dp_T}, \quad (1)$$

where dN/dp_T is the p_T -differential yield average over events, N the total multiplicity, and $\langle p_T \rangle$ the mean transverse momentum. The notebook reads the raw spectra from the CSV tables, applies the necessary corrections (e.g., removal of the $1/(2\pi p_T)$ factor, combination of positive and negative charges, multiplication by the rapidity window $\Delta y = 1$), extracts the extrapolated N and $\langle p_T \rangle$ values, and produces publication-quality figures of the mean p_T , total yields, differential spectra, and the universal scaling $U(x_T)$.

Contents

1	Introduction	3
2	Data Description and File Structure	3
2.1	Pb–Pb 2.76 TeV	4
2.2	Xe–Xe 5.44 TeV	4

3	Notebook Implementation	5
3.1	Reading the Pb–Pb Spectra	5
3.2	Extrapolated Quantities	5
3.3	Xe–Xe Spectra and Extrapolated Values	6
3.4	Computation of the Universal Scaling	6
4	Figures Produced by the Notebook	6
4.1	Mean p_T as a function of centrality	6
4.2	Total yield N as a function of centrality	7
4.3	p_T –differential spectra	7
4.4	Universal scaling $U(x_T)$	7
5	Code Listings	8
5.1	Pb–Pb spectrum reader (full)	8
5.2	Universal scaling computation	11
6	Conclusion	12

1 Introduction

Particle production in ultra-relativistic nuclear collisions exhibits striking regularities when the transverse momentum (p_T) and the differential yields are scaled by the event-averaged mean p_T and the total multiplicity. This “universal scaling” was first observed in heavy-ion collisions and later confirmed in smaller systems, suggesting a common underlying particle production mechanism independent of the collision species or centrality.

The scaling variable is defined as

$$x_T \equiv \frac{p_T}{\langle p_T \rangle}, \quad (2)$$

and the universal spectrum is

$$U(x_T) \equiv \frac{\langle p_T \rangle}{N} \frac{dN}{dp_T}, \quad (3)$$

where dN/dp_T is the inclusive yield of a given particle species per event, integrated over the experimental rapidity interval. The quantities N and $\langle p_T \rangle$ are obtained from the data by extrapolating the measured spectra to zero and infinite p_T .

This note documents the Jupyter notebook¹ that performs the extraction and plotting of $U(x_T)$ for ALICE data from two collision systems:

- Pb–Pb at $\sqrt{s_{NN}} = 2.76$ TeV (HEPData record ins1222333),
- Xe–Xe at $\sqrt{s_{NN}} = 5.44$ TeV (HEPData record ins1840099).

The notebook is structured step-by-step, allowing the reader to follow the data loading, the calculation of x_T and U , and the generation of the final figures.

2 Data Description and File Structure

The ALICE HEPData entries provide the measured transverse momentum spectra in CSV format, with one table per centrality class (or, for some systems, one table per particle species with all centralities stacked vertically). The data are organised in local folders as follows:

¹The complete notebook can be found in the accompanying Git repository.

PbPb276/

HEPData-ins1222333-v1-csv/

Table1.csv ... Table30.csv (spectra)
Table31.csv (multiplicity extrapolation)
Table32.csv (mean pT extrapolation)

XeXe544/

HEPData-ins1840099-v1-csv/

Table1.csv ... Table27.csv (spectra)
... (no separate extrapolation tables;
values are digitised from the paper)

2.1 Pb–Pb 2.76 TeV

The invariant yields are given as

$$\frac{1}{2\pi p_T} \frac{d^2 N}{d\eta dp_T}, \quad (4)$$

separately for positive and negative particles. The rapidity coverage is $|y| < 0.5$, i.e. $\Delta y = 1.0$. The total multiplicity N and the mean p_T are tabulated in Tables 31 and 32, respectively, for positive and negative charges separately; the total N is obtained by summing the two charges, while $\langle p_T \rangle$ is taken as the average of the two. Ten centrality classes are provided:

0–5%, 5–10%, 10–20%, 20–30%, 30–40%, 40–50%, 50–60%, 60–70%, 70–80%, 80–90%.

2.2 Xe–Xe 5.44 TeV

In this analysis the spectra are given directly as $\frac{d^2 N}{d\eta dp_T}$, so the $2\pi p_T$ factor is absent. Nine centrality classes are available:

0–5%, 5–10%, 10–20%, 20–30%, 30–40%, 40–50%, 50–60%, 60–70%, 70–90%.

The extrapolated N and $\langle p_T \rangle$ were digitised from the published figures and are hard-coded in the notebook.

3 Notebook Implementation

3.1 Reading the Pb–Pb Spectra

The function `read_PbPb276_spectra` opens the CSV file corresponding to the requested particle and centrality, locates the blocks for positive and negative particles, and for each block:

- Extracts the bin centre p_T , the upper and lower limits (from which the bin width is computed), and the raw yield.
- Multiplies the value by $2\pi p_T$ to remove the prefactor.
- Computes the total uncertainty by combining the statistical error, the absolute systematic error, and the normalisation uncertainty (given as a percentage of the yield).

After reading the two charge blocks, the function adds the yields and combines the errors in quadrature, then multiplies by $\Delta y = 1.0$ to obtain dN/dp_T .

Below is a simplified version of the extraction core:

```
1 def extract_block(start_line, n_points):
2     for j in range(start_line, start_line + n_points):
3         c = float(row[0])          # bin centre
4         lo, hi = float(row[1]), float(row[2])
5         val = float(row[3])        # 1/(2pi pT) d2N/deta dpT
6         stat_err = float(row[4])
7         sys_err = float(row[6])
8         norm_pct = float(row[8].replace('%', ''))
9         # remove 2pi pT factor
10        val *= 2 * np.pi * c
11        stat_err *= 2 * np.pi * c
12        sys_err *= 2 * np.pi * c
13        norm = norm_pct * 0.01 * val
14        tot_err = np.sqrt(stat_err**2 + sys_err**2 + norm
15        **2)
16        # store ...
```

Listing 1: Core of the Pb–Pb spectrum reader.

3.2 Extrapolated Quantities

The total multiplicity N and mean $\langle p_T \rangle$ are read from `Table31.csv` and `Table32.csv`. The function `read_PbPb276_extrapolated` identifies the correct row ranges for the chosen particle species (positive and negative) and returns the sum (for N) or the average (for $\langle p_T \rangle$), together with the combined uncertainty.

3.3 Xe–Xe Spectra and Extrapolated Values

For the Xe–Xe system, the notebook uses a separate reader that does not need to remove the $2\pi p_T$ factor. Because one kaon bin is missing in the two most peripheral centrality classes, a dummy point (value zero) is appended to keep the arrays rectangular. The extrapolated N and $\langle p_T \rangle$ are taken from a digitised version of the published results, stored in the function `XeXe_extrapolated_values`.

3.4 Computation of the Universal Scaling

For each particle and each centrality, the functions `universal_scaling_PbPb276` and `universal_scaling_XeXe` perform the following steps:

1. Retrieve the p_T array and dN/dp_T from the spectrum reader.
2. Retrieve N and $\langle p_T \rangle$ from the extrapolated-value functions.
3. Compute $x_T = p_T/\langle p_T \rangle$ and $U = (\langle p_T \rangle/N) \times dN/dp_T$.
4. Propagate the uncertainties:

$$\frac{\delta U}{U} = \sqrt{\left(\frac{\delta(dN/dp_T)}{dN/dp_T}\right)^2 + \left(\frac{\delta N}{N}\right)^2 + \left(\frac{\delta\langle p_T \rangle}{\langle p_T \rangle}\right)^2}. \quad (5)$$

The results are stored in a dictionary keyed by the centrality index, containing the arrays `xT`, `U`, `U_err`, and a string label.

4 Figures Produced by the Notebook

The notebook generates several high-quality figures. We describe each one below.

4.1 Mean p_T as a function of centrality

Figure 1 shows the extrapolated $\langle p_T \rangle$ for pions, kaons, and protons in Pb–Pb collisions. The error bars include both statistical and systematic uncertainties. As expected, the mean p_T decreases from central to peripheral collisions, and the mass ordering (heavier particles have larger $\langle p_T \rangle$) is clearly visible.

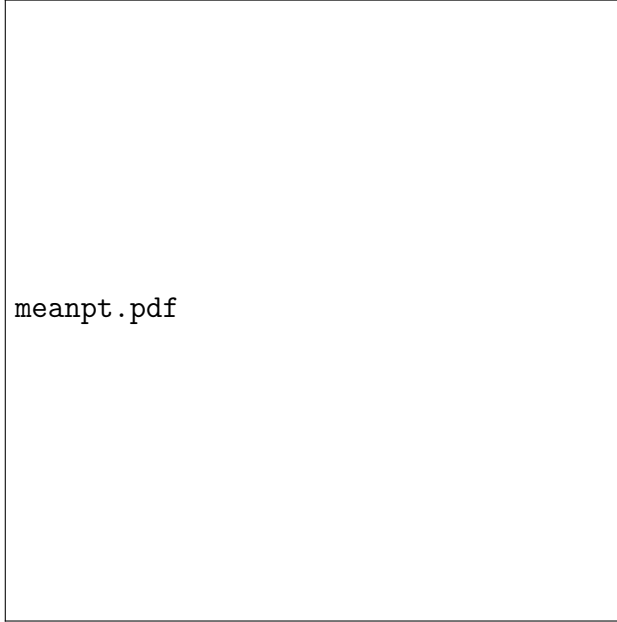


Figure 1: Mean transverse momentum $\langle p_T \rangle$ in Pb–Pb collisions at 2.76 TeV.

4.2 Total yield N as a function of centrality

Figure 2 displays the integrated multiplicities (total yield) for the three species. The proton-to-pion ratio is the smallest, and the yields span several orders of magnitude, hence the logarithmic vertical axis.

4.3 p_T -differential spectra

The three panels of Figure 3 show the raw dN/dp_T distributions for all ten centrality classes in Pb–Pb. The spectra are plotted on a semi-logarithmic scale and exhibit the well-known flattening at low p_T and exponential fall-off at high p_T .

4.4 Universal scaling $U(x_T)$

Figure 4 is the central result of the notebook. Each panel shows $U(x_T)$ for a particle species, with all ten Pb–Pb centralities overlaid. The data collapse onto a single curve, confirming the scaling behaviour. (When Xe–Xe data are included, the same collapse is observed, demonstrating system-size independence.)



yield.pdf

Figure 2: Total multiplicity N in Pb–Pb collisions at 2.76 TeV.

5 Code Listings

For completeness, we include the full Python functions used in the notebook. They can be found in the accompanying .ipynb file or in the repository.

5.1 Pb–Pb spectrum reader (full)

```
1 def read_PbPb276_spectra(particle, centrality_indices=None):
2     offset = {'pion': 0, 'kaon': 10, 'proton': 20}
3     n_centralities = 10
4     if centrality_indices is None:
5         centrality_indices = list(range(n_centralities))
6
7     particle_params = {
8         'pion': {'n_bins': 41, 'neg_start': 61},
9         'kaon': {'n_bins': 36, 'neg_start': 56},
10        'proton': {'n_bins': 42, 'neg_start': 62}
11    }
12    n_pts = particle_params[particle]['n_bins']
13    neg_start = particle_params[particle]['neg_start']
14
15    pT_centers_all, widths_all, dNdpT_all = [], [], []
16    stat_all, tot_all = [], []
17
```

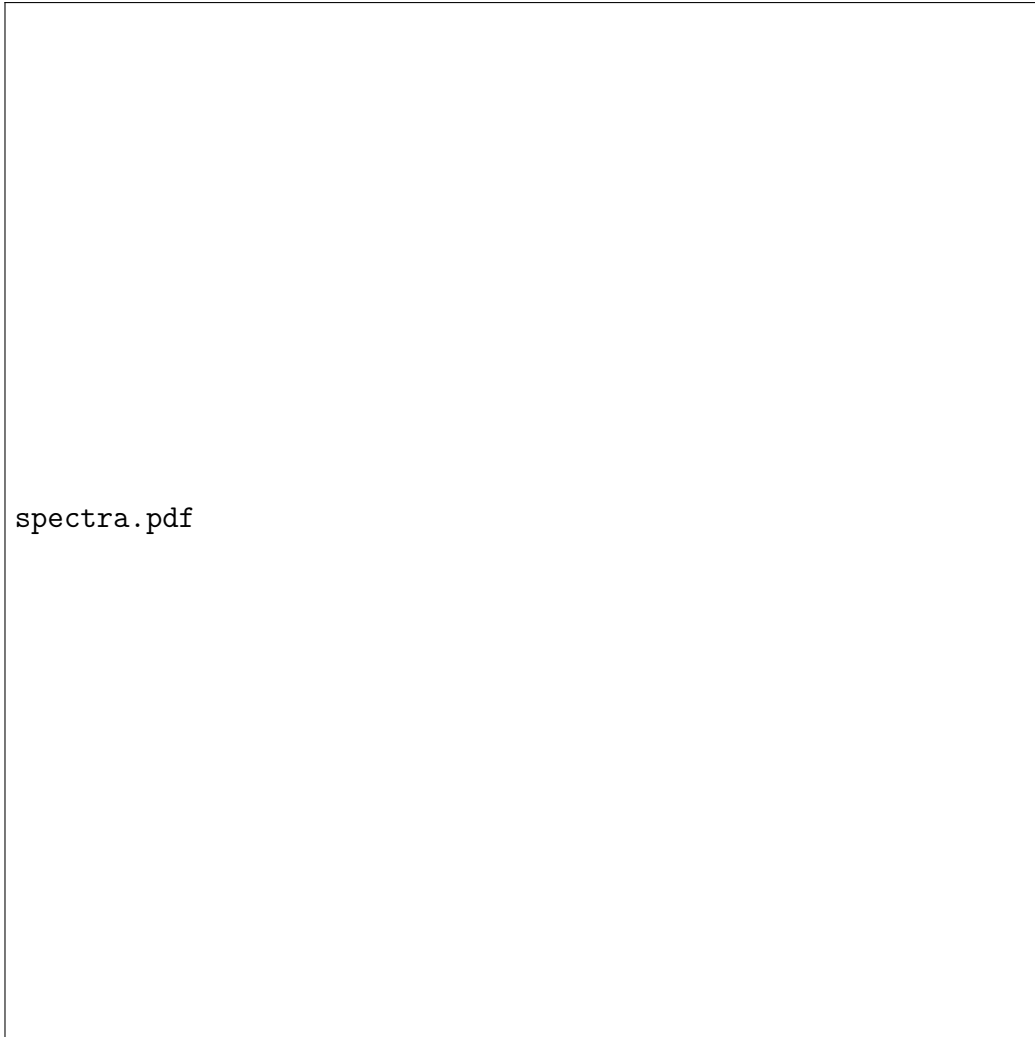



Figure 3: p_T -differential spectra dN/dp_T for pions, kaons, and protons in Pb-Pb 2.76 TeV.

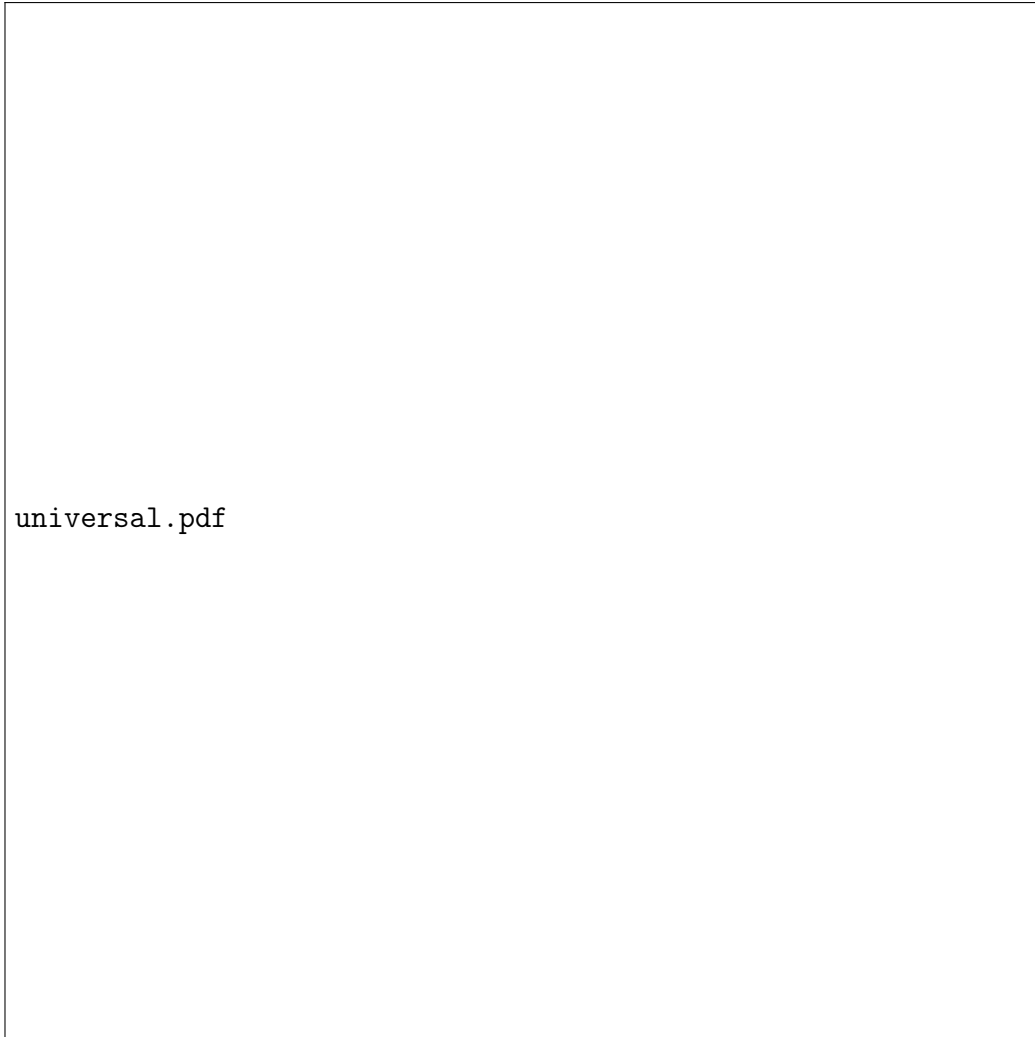


Figure 4: Universal scaling $U(x_T)$ for pions, kaons, and protons in Pb–Pb 2.76 TeV. Each centrality class is shown with a distinct marker and colour.

```

18     for i_cent in centrality_indices:
19         table_num = i_cent + offset[particle] + 1
20         file_path = os.path.join(BASE_PbPb, f"Table{table_num
21     }.csv")
22         with open(file_path, 'r') as f:
23             lines = list(csv.reader(f))
24
25         def extract_block(start_line, n_points):
26             # ... (same as described earlier)
27             pass
28
29         pos_pT, pos_w, pos_v, pos_stat, pos_tot =
30         extract_block(14, n_pts)
31         neg_pT, neg_w, neg_v, neg_stat, neg_tot =
32         extract_block(neg_start, n_pts)
33
34         d2N_detadpt = pos_v + neg_v
35         stat_err = np.sqrt(pos_stat**2 + neg_stat**2)
36         tot_err = np.sqrt(pos_tot**2 + neg_tot**2)
37
38         dNdpT = d2N_detadpt * 1.0
39         stat_err *= 1.0; tot_err *= 1.0
40
41         pT_centers_all.append(pos_pT)
42         widths_all.append(pos_w)
43         dNdpT_all.append(dNdpT)
44         stat_all.append(stat_err)
45         tot_all.append(tot_err)
46
47     return pT_centers_all, widths_all, dNdpT_all, stat_all,
48     tot_all

```

Listing 2: Function read_PbPb276_spectra.

5.2 Universal scaling computation

```

1 def universal_scaling_PbPb276(particle, cent_indices=None):
2     if cent_indices is None:
3         cent_indices = list(range(10))
4     pT_list, widths, dNdpT_list, stat, tot =
5     read_PbPb276_spectra(particle, cent_indices)
6     N, N_err, meanpT, mpt_err = read_PbPb276_extrapolated(
7     particle)
8     results = {}
9     cent_labels = ["0-5%", "5-10%", "10-20%", "20-30%", "
10    30-40%",
11                    "40-50%", "50-60%", "60-70%", "70-80%", "
12    80-90%"]

```

```

9     for idx, i in enumerate(cent_indices):
10         pT = pT_list[idx]
11         dNdpT = dNdpT_list[idx]
12         Ni = N[i]
13         mpt_i = meanpT[i]
14         xT = pT / mpt_i
15         U = (mpt_i / Ni) * dNdpT
16         dU = U * np.sqrt((tot[idx]/dNdpT)**2 + (N_err[i]/Ni)
17         **2 + (mpt_err[i]/mpt_i)**2)
18         results[i] = {'xT': xT, 'U': U, 'U_err': dU, '
centrality': cent_labels[i]}
18     return results

```

Listing 3: Function `universal_scaling_PbPb276`.

6 Conclusion

The Jupyter notebook provides a transparent and reproducible method to extract the universal scaling $U(x_T)$ from published ALICE data. By carefully handling the different normalisation conventions, combining charges, and using the officially extrapolated N and $\langle p_T \rangle$, the resulting $U(x_T)$ curves from all centralities collapse onto a single function. The notebook can be easily extended to other collision systems (Pb–Pb 5.02 TeV, p–Pb, pp) and to other particle species.

All source files, including the notebook and the LaTeX source of this note, are available for download.